

assignment6

DSA Assignment #6: 🌲 (1/3)

Updated 2026-04-05 21:54 GMT+8

Compiled by 徐前 物理学院 (2026 Spring)

说明:

1. 解题与记录:

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排:** 提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交:** 如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E94.二叉树的中序遍历

dfs, stack, <https://leetcode.cn/problems/binary-tree-inorder-traversal/>

思路:

按照要求模拟。

代码:

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
```

```
def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
    stack = [(root, 0)]
    ans = []
    while stack:
        node, color = stack.pop()
        if not node: continue
        if color:
            ans.append(node.val)
        else:
            stack.append((node.right, 0))
            stack.append((node, 1))
            stack.append((node.left, 0))
    return ans
```

代码运行截图（至少包含有"Accepted"）



E108.将有序数组转换为二叉搜索树

<https://leetcode.cn/problems/convert-sorted-array-to-binary-search-tree/>

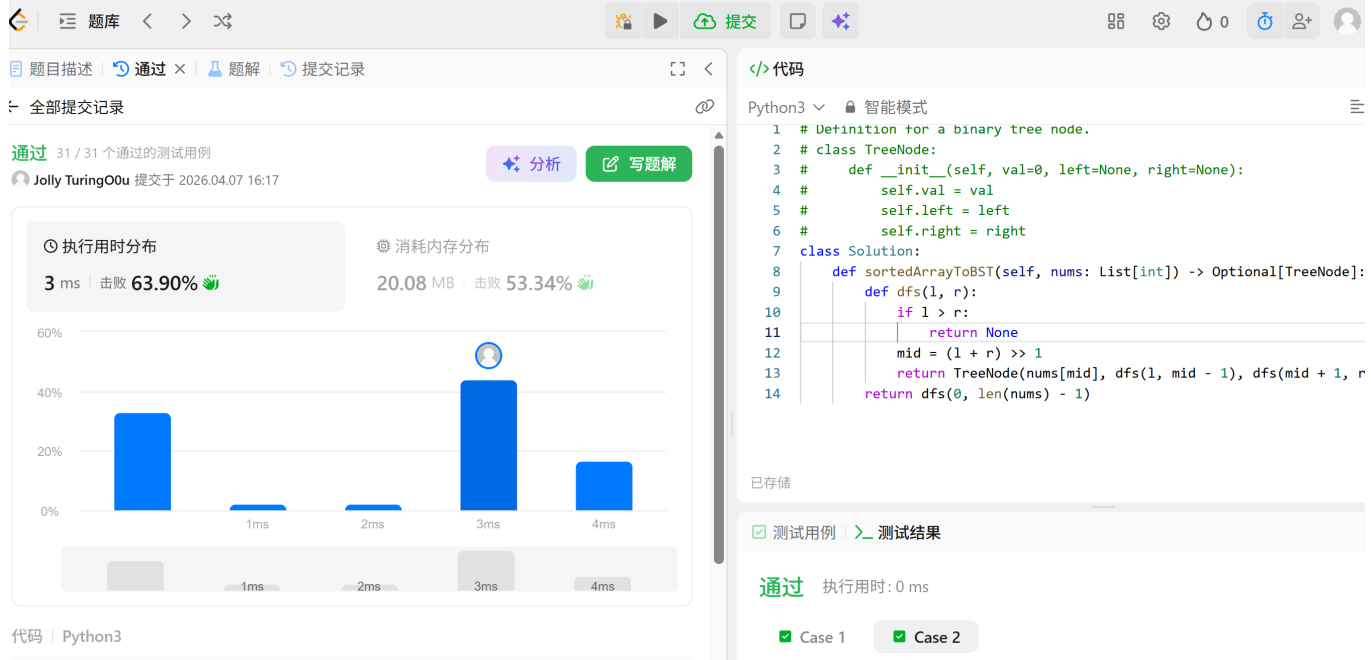
思路：

递归求解，每次把数组一分为二。递归边界是数组长度为0，返回空值。

代码：

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def dfs(l, r):
            if l > r:
                return None
            mid = (l + r) >> 1
            return TreeNode(nums[mid], dfs(l, mid - 1), dfs(mid + 1, r))
        return dfs(0, len(nums) - 1)
```

代码运行截图（至少包含有"Accepted"）



M102.二叉树的层序遍历

bfs, <https://leetcode.cn/problems/binary-tree-level-order-traversal/>

思路:

用BFS，为了实现分层遍历，可以额外记录深度d，更好的做法是每次记录队列的长度n，然后在一次遍历中只访问前n个节点。

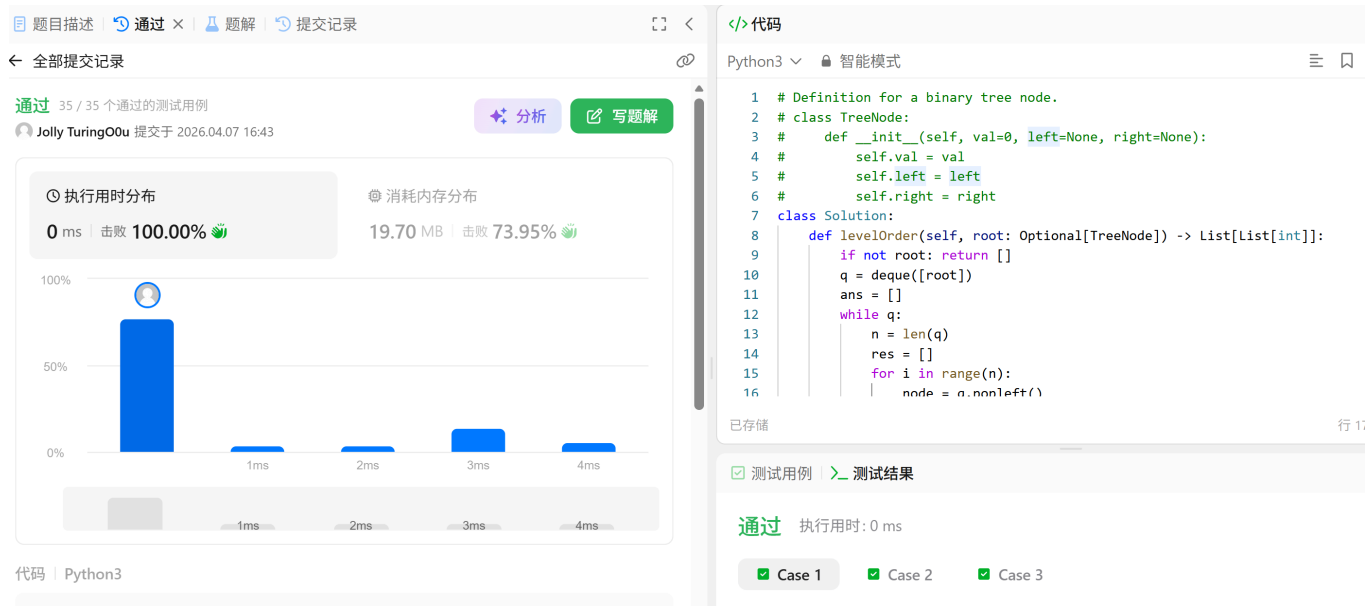
代码:

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if not root: return []
        q = deque([root])
        ans = []
        while q:
            n = len(q)
            res = []
            for i in range(n):
                node = q.popleft()
                if node.left: q.append((node.left))
                if node.right: q.append((node.right))
                res.append(node.val)
            ans.append(res)
        return ans

```

代码运行截图（至少包含有"Accepted"）



M1123.最深叶节点的最近公共祖先

dfs, <https://leetcode.cn/problems/lowest-common-ancestor-of-deepest-leaves/>

思路:

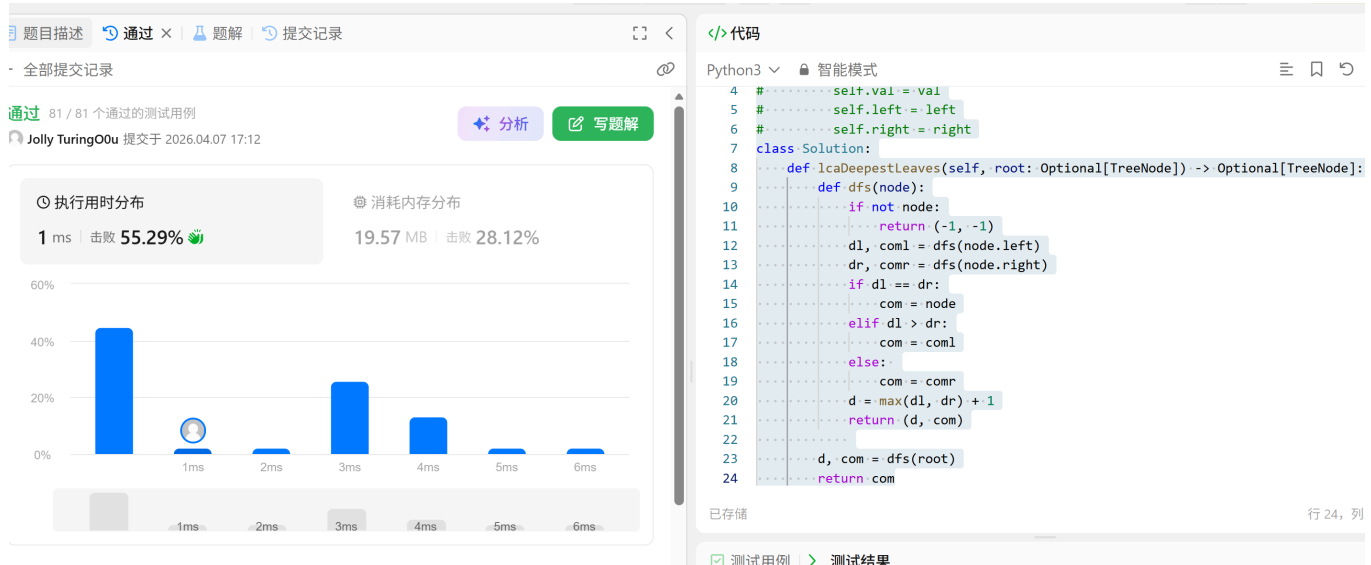
递归处理每个节点对应的数，返回树的深度d和LCA，分别考虑左子树的深度dl和右子树的深度dr，深度不同时取更深的子树的LCA继承，如果一样深，那么各自至少有一个最深的叶节点，共同祖先为当前节点node。

代码：

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def lcaDeepestLeaves(self, root: Optional[TreeNode]) ->
Optional[TreeNode]:
    def dfs(node):
        if not node:
            return (-1, -1)
        dl, coml = dfs(node.left)
        dr, comr = dfs(node.right)
        if dl == dr:
            com = node
        elif dl > dr:
            com = coml
        else:
            com = comr
        d = max(dl, dr) + 1
        return (d, com)

    d, com = dfs(root)
    return com
```

代码运行截图（至少包含有"Accepted"）



M07161: 森林的带度数层次序列存储

tree, <http://cs101.openjudge.cn/practice/07161/>

思路:

使用双端队列, 根据带度层序遍历构建树, 然后再做后序遍历。

代码

```
from collections import deque

class TreeNode:
    def __init__(self, ch):
        self.ch = ch
        self.children = []

def postorder(root):
    res = []
    def dfs(node):
        if node is None:
            return
        for child in node.children:
            dfs(child)
        res.append(node.ch)
    dfs(root)
    return res

def main():
    n = int(input())
    root = TreeNode('')
```

```

for _ in range(n):
    line = input().split()
    node = TreeNode(line[0])
    root.children.append(node)
    q = deque([(node, int(line[1]))])
    i = 2
    while i < len(line):
        cur, d = q.popleft()
        for j in range(d):
            nxt = TreeNode(line[i])
            cur.children.append(nxt)
            q.append((nxt, int(line[i + 1])))
            i += 2

print(' '.join(postorder(root)).strip())

if __name__ == '__main__':
    main()

```

(至少包含有"Accepted")

#52224655提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```

from collections import deque

class TreeNode:
    def __init__(self, ch):
        self.ch = ch
        self.children = []
    def postorder(self, root):
        res = []
        def dfs(node):
            if node is None:
                return
            for child in node.children:
                dfs(child)
            res.append(node.ch)
        dfs(root)
        return res

```

基本信息

#: 52224655
 题目: 07161
 提交人: 25n2500011619(xq)
 内存: 3700kB
 时间: 27ms
 语言: Python3
 提交时间: 2026-04-07 18:00:47

M27928: 遍历树

adjacency list, dfs, <http://cs101.openjudge.cn/practice/27928/>

思路:

用邻接表存每个节点的子节点，并对children作排序，用哈希表建立值到序号的对应，从入度为0的根节点进行DFS遍历，遍历顺序按照该节点和所有子节点的值由小到大进行。

代码

```

from bisect import bisect_left
from typing import Optional

class TreeNode:
    def __init__(self, x=0):
        self.val = x
        self.children = []

def main():
    n = int(input())
    rank = {}
    node = []
    d = [0] * n
    for i in range(n):
        data = list(map(int, input().split()))
        rank[data[0]] = i
        cur = TreeNode(data[0])
        cur.children = sorted(data[1:])
        node.append(cur)
    for i in range(n):
        for j in node[i].children:
            d[rank[j]] += 1
    root = node[d.index(0)]
    res = []

    def dfs(current: Optional[TreeNode]) -> None:
        if current is None:
            return
        if current.children:
            idx = bisect_left(current.children, current.val)
            for _ in range(idx):
                dfs(node[rank[current.children[_]]])
            res.append(current.val)
            for _ in range(idx, len(current.children)):
                dfs(node[rank[current.children[_]]])
        else:
            res.append(current.val)

    dfs(root)
    print(*res, sep='\n')

if __name__ == '__main__':
    main()

```


(至少包含有"Accepted")

#52225700提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: **Accepted**

源代码

```
from bisect import bisect_left
from typing import Optional

class TreeNode:
    def __init__(self, x=0):
        self.val = x
        self.children = []

def main():
    n = int(input())
    rank = {}
    node = []
    d = [0] * n
    for i in range(n):
        data = list(map(int, input().split()))
        rank[data[0]] = i
        cur = TreeNode(data[0])
        cur.children = sorted(data[1:])
        node.append(cur)
```

基本信息

#: 52225700
题目: 27928
提交人: 25n2500011619(xq)
内存: 4404kB
时间: 35ms
语言: Python3
提交时间: 2026-04-07 20:01:08

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2026spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

做了一些每日选作的题目，参加了LeetCode 496 虚拟周赛，AC4。

题目	最新结果	总提交数
✓ 3892. 产生至少 K 个峰值的最少操作次数 困难	通过	5 ✓
✓ 3891. 最大化特殊下标数目的最少增加次数 中等	通过	2 ✓
✓ 3890. 可由多种立方和构造的整数 中等	通过	1 ✓
✓ 3889. 镜像频次距离 中等	通过	1 ✓
✓ 3740. 三个相等元素之间的最小距离 I 简单	通过	1 ✓
✓ 3741. 三个相等元素之间的最小距离 II 中等	通过	1 ✓
✓ 874. 模拟行走机器人 中等	通过	2 ✓
✓ 297. 二叉树的序列化与反序列化 困难	通过	1 ✓

3892：产生至少k个峰值的最少操作次数

DP, <https://leetcode.cn/problems/minimum-operations-to-achieve-at-least-k-peaks/>

这道题结合了环形数组与DP，与213. 打家劫舍II类似，需要把环断成链然后额外判断首末的情况，同时本题在时间和空间上都卡常数，需要预处理每个位置的操作次数，并使用滚动数组优化空间。

代码：

```
class Solution:
    def solve(self, nums: list[int], k: int) -> int:
        def f(i):
            mx = max(nums[i - 1], nums[i + 1])
            return max(mx + 1 - nums[i], 0)

        n = len(nums)
        inf = 10 ** 9
        dp = [[[inf, inf] for j in range(k + 1)] for i in range(2)]
        fx = [f(i) for i in range(1, n - 1)]
        dp[0][0][0] = 0
```

```

    for i in range(1, n - 1):
        i0 = i & 1
        for j in range(min((i + 1) // 2, k) + 1):
            dp[i0][j][0] = min(dp[i0 ^ 1][j])
            if j:
                dp[i0][j][1] = dp[i0 ^ 1][j - 1][0] + fx[i - 1]
    return min(dp[(n - 2) & 1][k])

def minOperations(self, nums: List[int], k: int) -> int:
    n = len(nums)
    if k > n // 2:
        return -1
    cnt = 0
    for i in range(n):
        if nums[i - 1] < nums[i] > nums[(i + 1) % n]:
            cnt += 1
    if k <= cnt:
        return 0
    ans1 = self.solve([nums[-1]] + nums, k)
    ans2 = self.solve(nums + [nums[0]], k)
    return min(ans1, ans2)

```